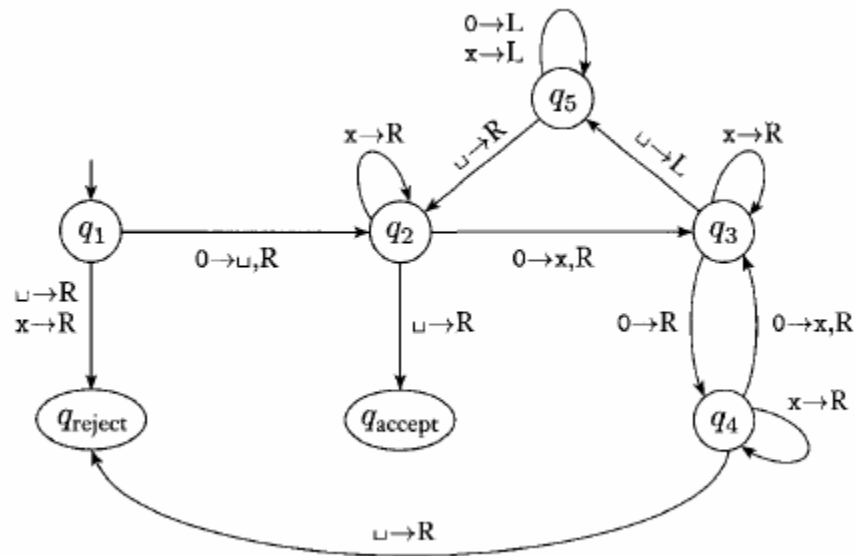
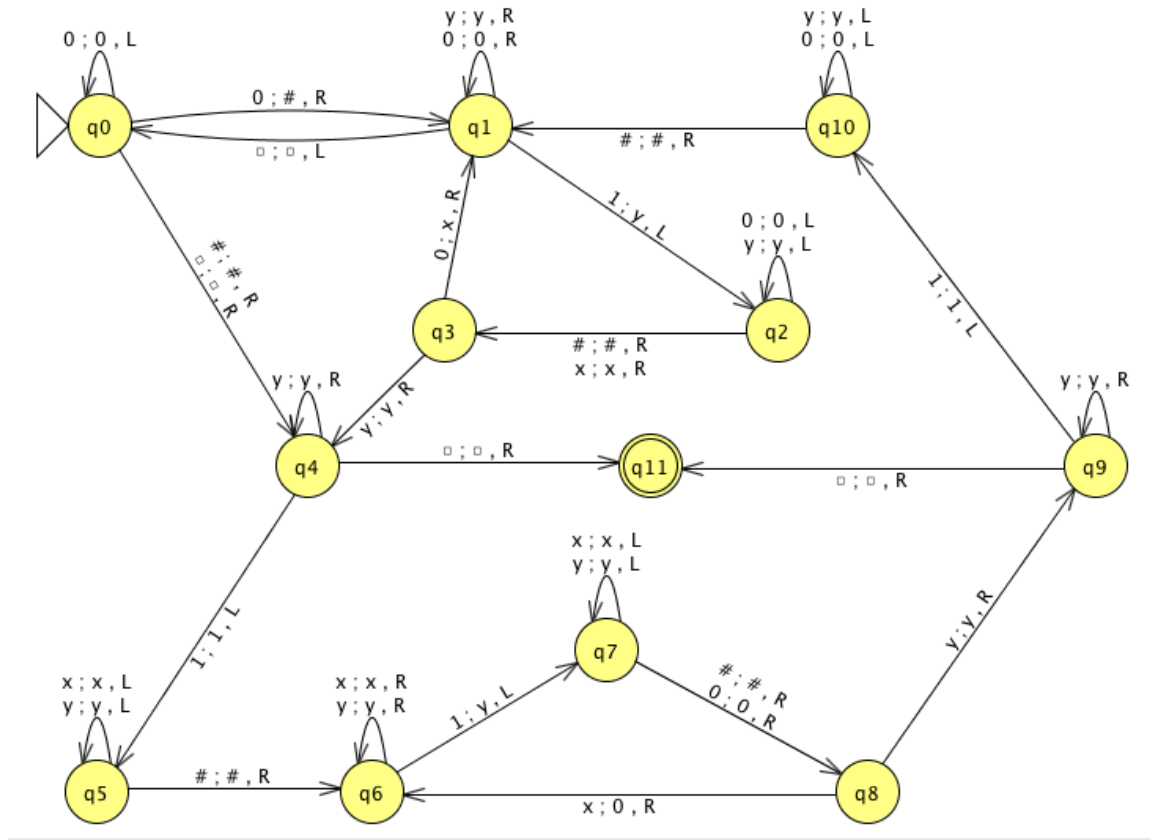


1. List the sequence of configurations that the following TM will go through on input 000. The "boxy U" character here is denoting empty tape cells



- 000u
- $q_1 000u$
- $uq_2 00u$
- $uxq_3 0u$
- $0x0q_4 u \rightarrow q_{\text{reject}}$

2. Draw a TM for the language $L = \{0^i 1^j \mid j \text{ is divisible by } i, i, j \geq 0\}$. Include a brief explanation that will help me understand how your TM works. You might either briefly describe each of your states, or explain the purpose of each block of states.



The TM construction, for the language $L = \{0^i 1^j \mid j \text{ is divisible by } i \text{ and } i, j \geq 0\}$, was constructed based on the following three cases:

1. First case, $|0^i| = |1^j|$ where $i, j \geq 0$ and $i=j$.
2. Second case, $|0^i| < |1^j|$ where $i, j \geq 0$ and $i < j$.
3. Third case, $|0^i| > |1^j|$ where $i \geq 0$, and $j = 0$.

These three cases are described with details below:

1. In the first case, since i will always be equal to j , j will always be divisible by i . This case was addressed in the following manner:

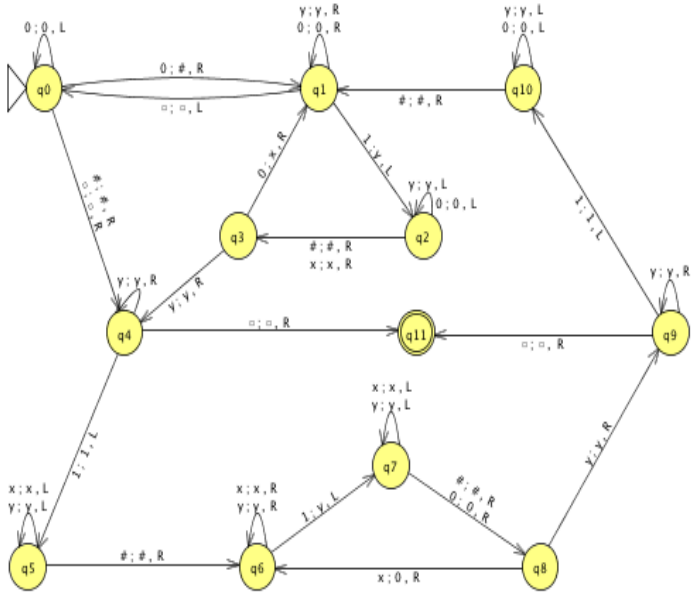
At state q_0 , mark the first 0 with a unique symbol on the tape in order to know when the first 0 has been reached. At state q_1 , skip all 0's till a 1 is found and mark this on the tape with 1. Then, in state q_2 skip all 0's and y's to the left till the first 0 has been reached. Once this 0 has been reached, proceed to mark each remaining 0 with x by repeating the same process at state q_1 and q_2 with the exception that this time you will be moving to the left of the tape till the last x you marked is reached instead of the first 0. Finally, at state q_3 , if you do not have any remaining 0's it will go to state q_4 and test whether the number of 0's and y's are even or not. If so, the empty symbol will be reached and therefore accept. Otherwise, it will test second case.

2. In the second case, j will be divisible by i if and only if j is a multiple of i . This case was addressed, in my TM construction, through the following algorithm:

At state q_4 , if we still have any remaining 1's the TM will take the following path to test whether j is a multiple of i or not. At state q_5 it moves all the way to the left till the unique symbol for the first 0 is reached. Once this symbol is reached, it proceeds to do a similar approach to case 1 with the only difference that instead of marking 0's to x's we will be marking x's to 0's. This approach will allow to move back and forth from case 1 to case 2 and from case 2 to case 1 infinitely number of times, allowing us to accept infinitely multiple numbers of i , which is represented as j . The switch transition is done in states q_8 and q_9 as follows. If in state q_8 no more x's are remaining it will go to state q_9 to test if all 1's are marked. If so, then j , the number of ones, is a multiple of i , the number of 0's. Otherwise, it goes to state q_{10} where it moves left all the way till the unique symbol for the first 0 is reached to repeat case 1. This process is done over and over until the TM either rejects or accepts. The TM will reject if it gets stuck in any of the loop by having no transition to the last 1 remaining.

3. In the third case, when $j = 0$ we can have any number of 0's since 0 divided by any number > 0 will always be 0. This is addressed, on my TM construction, in the following manner. At state q_1 , if you move all the way to the right and no 1 is found you will reach the empty symbol. Once the empty symbol is reached, the TM goes back to state q_0 where it checks if there are only 0's left by moving all the way to the left till the first 0 was reached. Notice that this will only accept if all we have left are 0's. At state q_0 the machine will not move to the left if any 1 was found before the empty string, and therefore the machine will be stuck at state q_0 forcing the TM to reject such input.

Testing TM



Input	Result
00111	Reject
0011	Accept
	Accept
1	Reject
01	Accept
001111	Accept
0011111	Reject
00111111	Accept
000111111	Accept
0	Accept
000000	Accept

Zooming Output Example:

Input	Result
00111	Reject
0011	Accept
	Accept
1	Reject
01	Accept
001111	Accept
0011111	Reject
00111111	Accept
000111111	Accept
0	Accept
000000	Accept

3. A Turing machine without left is just like a normal one except the transition function has the form $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{R, S\}$

Just answer the question "What class of languages do these machines recognize?"

The class of languages accepted by a $\{R, S\}$ Turing machine are all regular languages.

And justify your answer.

To show that a $\{R, S\}$ Turing machine accepts only regular languages we could base on the fact that $\{R, S\}$ TM is not equivalent to a standard Turing by showing a language accepted by a standard TM and not by a $\{R, S\}$ TM. A $\{R, S\}$ TM has no memory because it cannot move backwards on the tape. Therefore, we will prove that the standard Turing machine is stronger than the $\{R, S\}$ Turing machine.

1- Example:

Let $L = \{0^n 1^n \mid n > 0\}$ be Turing recognizable. We can see that is not possible to write a $\{R, S\}$ Turing machine that recognizes L because we are not able to move backwards and remember how many 0's we have read to ensure we read the same number of 1's.

2- Since $\{R, S\}$ cannot accept a CFL, then it certainly cannot accept any other RE languages other than regular languages. This can be proved by construction as follows:

Make a $\{R, S\}$ Turing machine simulates an NFA in the following manner:

- Let $M = \{Q, \Sigma, \gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}\}$ be the TM
- Let $N = \{Q', \Sigma, \gamma', \delta', q'_0, q'_{\text{accept}}, q'_{\text{reject}}\}$ be the NFA.

First, use NFA state machine to remember state of TM and current tape symbol to make up for $|Q \times \gamma|$ states. Next, simulate two moves of the $\{R, S\}$ TM,

- Stay put move is simulated using ϵ transition as follows:
If $\delta(q_1, a) = (q_2, b, S)$ then we add transition $\delta'((q_1, a), \epsilon) = (q_2, b)$
- Move right is simulated by reading a symbol from the input as follows:
If $\delta(q_1, a) = (q_2, b, R)$ then we add transition $\delta'((q_1, a), c) = (q_2, c)$ for all $c \in \Sigma$.

Finally, three states q'_{start} , q'_{accept} and q'_{reject} are added with following transitions.

$$Q' = Q \times \gamma \cup \{q'_{\text{start}}, q'_{\text{accept}}, q'_{\text{reject}}\}$$

$$\delta'(q'_{\text{start}}, a) = \{(q_0, a)\} \text{ for all } a \in \Sigma.$$

$$\delta'(q_{\text{accept}}, a) = \{q'_{\text{accept}}\} \text{ for all } a \in \Sigma.$$

$$\text{If } \delta(p, a) = (q_{\text{accept}}, b, m) \text{ where } m = R \text{ or } m = S \text{ we add } \delta'((p, a), \epsilon) = \{q_{\text{reject}}\}$$

$$\text{If } \delta(p, a) = (q_{\text{reject}}, b, m) \text{ where } m = R \text{ or } m = S \text{ we add } \delta'((p, a), \epsilon) = \{q_{\text{reject}}\}$$

The TM simulated an NFA, proving that a $\{R, S\}$ TM only accepts regular languages since it cannot accept languages of type $L = \{0^n 1^n \mid n > 0\}$ as shown in the first section.

4. Show that PDAs with two stacks are just as powerful as Turing machines.

To show that a 2-stack PDA is as powerful as a Turing machine we will simulate a standard Turing machine by using a 2-stack PDA by using the first stack to represent the portion of the tape to the left of the tape head, and the second stack to store the content of the tape to the right of the tape head. A 2-stack PDA is defined by a 6-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ where $Q, \Sigma, \Gamma, \delta$ are finite sets, $q_0 \in Q$, $F \subset Q$ and δ is a function from $Q \times \Sigma_\epsilon \times \Gamma_\epsilon \times \Gamma_\epsilon$ to $p(Q \times \Gamma_\epsilon \times \Gamma_\epsilon)$ where Σ_ϵ and Γ_ϵ denote the extended alphabets $\Sigma \cup \{\epsilon\}$ and $\Gamma \cup \{\epsilon\}$.

The components have the following meanings:

- Q is the set of states
- Σ is the input alphabet
- Γ is the stack alphabet, where both stacks use the same alphabet
- q_0 is the start state
- F is the set of accepting states
- $\delta(q, \alpha, x, y) = (r, w, z)$, where q is the current state, α is an input symbol, and the symbols on top of the two stacks x and y , possibly equal to the empty string. This returns the set of all possible triples (r, w, z) , where r is the new state, w and z are the symbols to be pushed on the stacks replacing x and y . Transitions between configurations are defined by the following rule $(q, a\alpha, xy1, yy2) \rightarrow (r, \alpha, wy1, xy2)$, if $(r, w, z) \in \delta(q, a, x, y)$.

Simulation:

First, the PDA pushes $\$$ on the two stacks to mark the bottoms. Next, before simulation, the PDA reads the entire input and pushes on the second stack. Then, the content of the second stack is moved to the first stack. The PDA is now in configuration $(q, \epsilon, w\$, \$)$ where w is the original input string. The configurations $(q, \epsilon, y1\$, y2\$)$ of the PDA can be used to represent the corresponding TM configurations $(y2^R, q, y1)$, where $y2^R$ is the reversal of string $y2$. The PDA can now simulate the TM, popping symbols from $y1$ and pushing them on $y2$ to move the tape head to the right, and popping from $y2$ and pushing on $y1$ to move left.

If the TM tries to move beyond the right margin, the second stack is left unchanged and a blank is pushed on the first stack. If the TM tries to move beyond the left margin, the second stack is left unchanged and the first symbol of the first stack is overwritten.

A computation on input w is a sequence of configurations $c_0, \dots, c_n$ such that c_0 is the start configuration $(q_0, w, \epsilon, \epsilon)$ and $c_{i-1} \rightarrow c_i$ for all $i = 1, \dots, n$. The computation is accepting if $c_n = (r, \epsilon, y1, y2)$ for some $r \in F$ and $y1, y2 \in \Gamma^*$. The language accepted by the automaton is defined as the set of all strings $w \in \Sigma^*$ such that $(q_0, w, \epsilon, \epsilon) \rightarrow^* (r, \epsilon, y1, y2)$ for some $r \in F$ and $y1, y2 \in \Gamma^*$. This shows that a 2-stack PDA can accept languages such as $L = \{a^n b^n c^n \mid n > 0\}$, proving this that a 2-stack PDA could be just as powerful as a TM.